

第 05 章 | 資料降維與特徵萃取實戰

學生版公開講義 | 請搭配本課程 Colab，依照段落逐步操作與自我檢查。

來源說明：課程參考《Python Machine Learning, 3rd Edition》；原始範例程式碼採 MIT License。本檔由恩恩統計家教整理為學生版學習摘要，不含原書 PDF 或掃描內容。

1. 本章學習路線

在面對現代大資料任務時，高維度資料（High-dimensional data）往往夾雜著冗餘資訊與雜訊。這不僅浪費儲存空間，更會引發「維度詛咒」（Curse of Dimensionality），使模型在稀疏的空間中難以捕捉泛化規律。降維技術（Dimensionality Reduction）的實務意義在於：透過特徵萃取（Feature Extraction）將原始資料映射至低維度子空間，在大幅壓縮資料體積的同時，保留最關鍵的資訊結構。

學習目標定義

本質差異：辨析「特徵選擇」（挑選原始維度）與「特徵萃取」（透過數學變換融合新維度）的不同。

演算法選型：透析 PCA（非監督、找變異）、LDA（監督、找分類）與 KPCA（非線性、找空間變形）的適用場景。

標準化之必要性：理解為何變異數計算極度依賴特徵標準化（Standardization）。

核心術語對照表

中文術語	英文術語	定義
主成分分析	Principal Component Analysis (PCA)	尋找資料最大變異方向的非監督式降維。
線性判別分析	Linear Discriminant Analysis (LDA)	最大化類別分離度（Separability）的監督式降維。
特徵對	Eigenpairs (Eigenvalues / Eigenvectors)	描述子空間「方向（向量）」與「重要性分數（值）」。
共變異數矩陣	Covariance Matrix	衡量多個特徵間連動關係（Correlation）的對稱矩陣。
解釋變異數比例	Explained Variance Ratio	PCA 中衡量個別主成分保留資訊量的百分比。
類別分離度	Class Separability	衡量降維後不同標籤類別是否「涇渭分明」的指標。

2. PCA：尋找資料的「最大公約數」

幾何直覺分析：茶壺剪影比喻

PCA 採取的「非監督式」實務不參考標籤，純粹觀察資料散佈。想像一個 3D 的「茶壺實體」，若要拍一張 2D 照片來記錄其外型，我們會尋找一個能涵蓋最大投影面積、輪廓最清晰的角度。

第一主成分 (PC1)：即投影後變異量最大（資料最分散）的方向。

第二主成分 (PC2)：必須與 PC1 完全垂直（Orthogonal），旨在捕捉剩下的最大變異，同時確保新維度間互不相關。

技術筆記：NumPy Solver 的陷阱

在計算特徵值時，請特別留意：

Pro-Tip: 實作時應優先選用 `scipy.linalg.eigh` 而非 `numpy.linalg.eig`。原因在於共變異數矩陣必定是對稱矩陣（Symmetric Matrix），`eigh` 專為對稱/赫米特矩陣設計，不僅運算效率更高，且能保證產出實數解，有效避免浮點數誤差導致的複數（Complex Numbers）問題，確保數值穩定性。

解釋變異數比例（Explained Variance Ratio）

PCA 的核心權衡（Tradeoff）在於：保留多少維度最划算？透過「累積階梯圖」（Cumulative Step Plot），我們可以量化資訊損失。

分析：在 Wine Dataset（13 維）中，通常 PC1 與 PC2 累計即可解釋約 60% 的總變異。這意味著我們捨棄了 11 個維度，卻保留了過半的資訊精華。

數學運作六大步驟

標準化：將特徵縮放至平均值為 0、變異數為 1。

計算共變異數矩陣 Σ ：量化特徵間的線性相關性。

分解特徵對：求解 $\Sigma v = \lambda v$ 。

排序：依特徵值 λ （變異量大小）由大到小排序。

建構轉換矩陣 W ：選取前 k 個特徵向量作為投影矩陣的列。

映射：計算 $X' = XW$ 得到降維後的資料。

3. LDA：最佳化類別分離度

分類最佳化分析

PCA 不使用標籤；LDA 則是監督式學習，會使用類別標籤，目標不是找最大變異，而是尋找最有利於分類的特徵子空間。若資料類別間差異細微，PCA 可能會因為保留了無關分類的變異而失靈，此時 LDA 才是更好的選擇。

核心邏輯：類別分離度（Class Separability）

LDA 追求「同類靠得近，異類離得遠」：

類別內散佈矩陣 (SW)：衡量同一類別內點的擠壓程度（越小越好）。

類別間散佈矩陣 (SB)：衡量不同類別重心（Mean Vectors）間的距離（越大越好）。

最佳化目標：求解 $SW^{-1}SB$ 的特徵對，最大化上述比率。

使用限制：維度極限與判別比例

維度上限：LDA 生成的線性判別量（Linear Discriminants）上限為 $c-1$ （ c 為類別數）。以 Wine Dataset 為例，即使原始有 13 個特徵，但因僅有 3 個類別，LDA 降維後最多只能產生 2 個成分。這是因為 SB 的秩（Rank）受限於類別重心數。

鑑別率（Discriminability Ratio）：類似 PCA 的變異數比例，LDA 使用判別比例來衡量各組向量對類別分離的貢獻。

scikit-learn 實戰提示

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA
lda = LDA(n_components=2)
# 核心關鍵：LDA 是監督式，訓練時必須傳入標籤 y_train
X_train_lda = lda.fit_transform(X_train_std, y_train)
# 測試集則僅需執行 transform，套用訓練集學到的投影矩陣
X_test_lda = lda.transform(X_test_std)
```

4. KPCA：解決非線性糾纏的「空間變形術」

非線性問題：當直線切不開資料時

若資料分佈如「半月形」（Half-moons）或「同心圓」，線性 PCA 會導致資料點在投影後高度重疊。KPCA 引入了核技巧（Kernel Trick），其實務是：先透過非線性映射 ϕ 將資料映射到較高維空間，在那裡資料會變得線性可分，再進行降維。

核心觀念：毛線球與超平面

想像糾結的「毛線球」（非線性資料）。KPCA 將其拋向空中（高維映射）並在瞬間將其「攤平」。在空中，我們可以用一張平坦的紙（線性超平面）輕易地將不同顏色的毛線分開。

關鍵技術差異

KPCA 與標準 PCA 不同：它並不建立轉換矩陣 W ，而是直接產出已投影好的資料點。這是因為在高維空間計算 $\phi(X)$ 成本極高，KPCA 透過核函數（如 RBF）直接計算樣本間的相似度矩陣 K 。

核心 Kernel 對照表

Kernel 類型	數學公式 $\kappa(x(i), x(j))$	技術特性與警告
RBF (高斯核)	$\exp(-\gamma \ x(i) - x(j)\ ^2)$	
Polynomial	$(x(i)^T x(j) + \theta)^p$	處理特定多項式關係。
Sigmoid	$\tanh(\eta x(i)^T x(j) + \theta)$	類似類神經網路的映射。

內存與效能預警

KPCA 是基於記憶體（Memory-based）的方法。

它需處理 $N \times N$ 的相似度矩陣。

量化風險：若樣本數 $N=100$ ，矩陣僅 10,000 元素；若 $N=10,000$ ，矩陣將膨脹至 108 元素，極易撐爆 Colab 記憶體。

5. 技術陷阱排除與操作守則

陷阱一：特徵標準化之必要

PCA/LDA 極度敏感於尺度。若特徵單位不同（如「公頃」與「公克」），變異數計算會被單位大的特徵主導，導致降維軸偏移。降維前必做 `StandardScaler`。

陷阱二：鏡像翻轉（Mirror Image）

你可能會困惑：「為什麼我的投影圖與講義左右相反？」

原因說明：這是正常的。特徵向量 v 與 $-v$ 在數學上是等價的（ $\sum \alpha v = \lambda \alpha v$ ，其中 $\alpha = \pm 1$ ）。底層解算器選擇正號或負號取決於實作版本（如 LAPACK），這不影響分類性能，僅是座標軸的正負取向不同。

常見錯誤三：KPCA 的 Gamma 設定不當

γ 決定了 RBF 的「勢力範圍」。設定過大會導致過度擬合（每個點自成一類）；設定過小則會過於平滑。建議先從 $\gamma=15$ （針對半月資料）觀察，再進行調整。

6. 課後總結與自我檢查

綜合對照表

特性	PCA	LDA	KPCA
監督類型	非監督 (Unsupervised)	監督 (Supervised)	非監督
線性度	線性	線性	非線性
核心目標	最大化變異量 (Variance)	最大化類別分離度	高維映射後降維
適用場景	資料壓縮、去噪	預處理分類任務	處理糾纏、非線性分佈

實作練習任務

變異量探索：在 Colab 執行 `PCA(n_components=None)`，印出 `explained_variance_ratio_`。

判斷：根據累積圖，若要保留 95% 資訊，Wine Dataset 至少需保留幾維？

邊界觀測：嘗試更改 KPCA 的 γ 值（由 0.1 調至 100），記錄不同 gamma 值造成的決策邊界變化。

自我檢查表

- ☐ 是否已對資料進行標準化？
- ☐ 是否理解 LDA 在 Wine 資料集中最多只能有 2 個成分？
- ☐ 在執行 LDA 時，`fit_transform` 是否正確傳入了 `y_train`？
- ☐ 是否能辨析為何 KPCA 會消耗大量記憶體？

本章總結：降維不只是為了視覺化，更是提升模型強健性的「去蕪存菁」。記住，雖然萃取出的成分失去了原始物理意義，但它們代表了資料最純粹的數學結構。掌握這三種工具，你就能應對從線性資料壓縮到非線性空間變換的所有挑戰。

\[END OF DOCUMENT\]